

Game Zone

Application pour commander des jeux vidéos

Projet réaliser dans le cadre de la présentation au

Titre Professionnel Développeur Web et Web Mobile

Présenté par

Alicia Chabane

1. Introduction

- 1.1 Présentation personnelle
- 1.2 Présentation du projet
- 1.3 Liste des compétences couvert par le projet

2.Cahier des charges

- 2.1 Besoins et objectifs de l'application
- 2.2 Public cibles
- 2.3 Fonctionnalités attendues

3. Environnement de travail et architecture du projet

- 3.1 Technologies utilisé
- 3.2 Arborescence

4.Conception Front-end

- 4.1 Wireframe user et admin
- 4.2 Maquette web et mobile

5.Conception Back-end (Merise)

- 5.1 MCD
- 5.2 MLD
- 5.3 MPD

6. UML et User story

7. Réalisation en front

- 7.1 pages
- 7.2 Composants réutilisable
- 7.3 Extrait code statique
- 7.4 Extrait code dynamique
- 7.5 Responsive
- 7.6 Règles d'accessibilité

8. Réalisation en back

- 8.1 Base de donnée
- 8.2 Création / migration
- 8.3 Création / model
- 8.4 Création / controller
- 8.5 Composants métier (logique serveur, POO)
- 8.6 Extraits de code (accès données)
- 8.7 Sécurisation côté client
- 8.8 Sécurisation côté serveur (auth, validation, etc.)

9.Procédure d'installation et de configuration

10.Jeu d'essai fonctionnel (seeders)

11. Déploiement

12.Conclusion

Introduction

Présentation personnelle

Je m'appelle Alicia Chabane, actuellement candidate au Titre Professionnel Développeur Web et Web Mobile. Passionnée par les nouvelles technologies et le monde du jeu vidéo, j'ai souhaité à travers ce projet mettre en pratique mes compétences en développement front-end et back-end. Ce projet m'a permis de renforcer mes connaissances en React, Laravel et Tailwind CSS, tout en découvrant les enjeux d'un site e-commerce complet.

Présentation du projet

Dans un contexte où le marché du jeu vidéo connaît une croissance exponentielle et où le commerce en ligne s'impose comme un mode d'achat privilégié, le projet Game Zone s'inscrit comme une réponse innovante aux attentes des gamers. Game Zone est une plateforme e-commerce spécialisée dans la vente de jeux vidéo, offrant un large catalogue couvrant les dernières nouveautés, les classiques incontournables et des éditions exclusives.

Notre ambition est de proposer aux passionnés de jeux vidéo une expérience d'achat simple, rapide et sécurisée, tout en mettant l'accent sur la qualité du service client et la diversité de notre offre. À travers ce projet, nous souhaitons créer une véritable communauté autour du jeu vidéo et devenir une référence incontournable dans le domaine de l'e-commerce ludique.

Liste des compétences couverte par le projet

Développer la partie front-end d'une application web ou web mobile sécurisé

1. Installer et configurer son environnement de travail en fonction du projet web ou web mobile
2. Maquetter une application
3. Réaliser des interfaces utilisateur statiques web ou web mobile
4. Développer la partie dynamique des interfaces utilisateur web ou web mobile

Développer la partie back-end d'une application web ou web mobile sécurisé

5. Créer une base de donnée
6. Développer les composants d'accès aux données
7. Développer la partie back-end d'une application web ou web mobile
8. Elaborer et mettre en oeuvre des composants dans une application de gestion de contenu ou e-commerce

Cahier des charges

Besoins et objectifs de l'application

Profil

- s'inscrire ou se connecter
- accéder au paramètres de son profil
- accéder à son panier
- et pouvoir se déconnecter

Gestion et vente de jeux vidéo

- Catalogue complet des jeux vidéo (nouveau, classiques, éditions spéciales)
- Gestion des stocks en temps réel
- Mise à jour régulière des fiches produits (descriptions, images,)

Expérience utilisateur et interface

- Navigation fluide et intuitive sur le site
- Recherche avancée par genre, plateforme, prix, popularité
- Système de filtres et de recommandations personnalisées

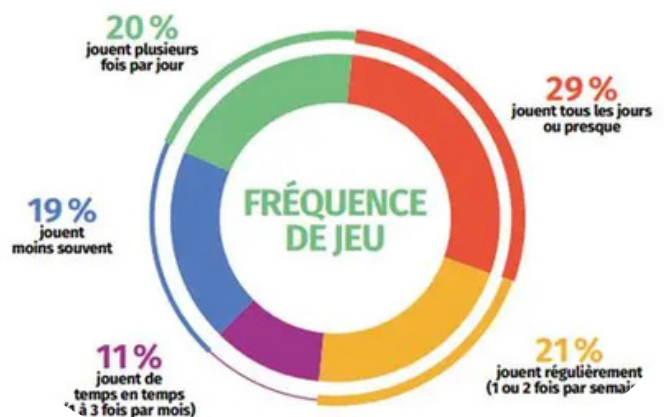
Gestion des commandes et paiements

- Processus de commande simple et sécurisé
- moyens de paiement (carte bancaire)
- Suivi des commandes et gestion des retours

Public cible

Joueurs adultes : Contrairement aux idées reçues, la majorité des joueurs sont des adultes.

Joueurs réguliers et occasionnels : Game Zone s'adresse à la fois aux passionnés qui achètent régulièrement des nouveautés et aux joueurs occasionnels qui recherchent des classiques ou des jeux accessibles à tous



MVP

Voici donc les fonctionnalités présentes dans notre MVP : l'utilisateur (ou administrateur) doit pouvoir s'authentifier avec un email et un mot de passe l'utilisateur doit pouvoir consulter les mentions légales et la politique de confidentialité du site

Parcour pour un User

- 1.Authentification : inscription et connexion sécurisées.
- 2.Consultation du catalogue : liste de jeux et fiches produits.
- 3.Panier simple : ajout et suppression de jeux.
- 4.Commande de base : validation et suivi (paiement simulé).

Parcour pour un Admin

- 1.L'admin se connecte et accède au dashboard.
- 2.Il consulte la liste des produits et peut en créer / modifier / supprimer.
- 3.Il consulte les commandes et peut mettre à jour leur statut.
- 4.Il gère les utilisateurs

Pages pour User

1. Accueil non connecter
2. Inscription ou Connexion
- 3..Accueil Connecter
4. Detail Produit
- 5.Panier
- 6.Payement
- 7.Mes Commandes

Pages pour Admin

1. Accueil non connecter
 2. Connexion
 - 3..Dashboard
 4. Gérer les produits
 - 5.Gérer les commandes
 - 6.Gérer les utilisateurs
 - 7.Créer un produit
-

Fonctionnalité principal

Fonctionnalité	Admin	User
Inscription	non	oui
Connexion	oui	oui
Accéder a son profil	oui	oui
Recherche de jeux	oui	oui
Choix de plateforme	quand il créer un jeux il peut choisir sur quelle plateforme le jeux sera	Sur quelle plateforme il veut le jeu
Panier	non	ajouter ou supprimer un ou plusieurs jeux de son panier
Gérer les produits	créer/modifier ou supprimer	non
Gérer les commandes	créer/modifier ou supprimer	non
Gérer les utilisateurs	créer/modifier ou supprimer	non

Environnement de travail et architecture du projet

Comme IDE j'ai utilisé VS Code sous windows en utilisant la stack Laravel, React, Inertia.js et Tailwind CSS, afin de mettre en place une architecture claire et performante reliant efficacement le front-end et le back-end.

Technologie utilisée

Backend :

Pourquoi :



Gestion robuste de la logique métier, des routes, de la base de données et de l'authentification.

Frontend :

Pourquoi :

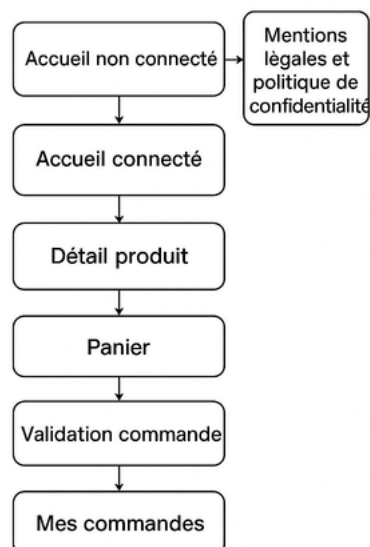


Structure modulaire, réactive et performante pour l'interface utilisateur. Framework CSS utilitaire pour un design moderne, responsive et facile à personnaliser.

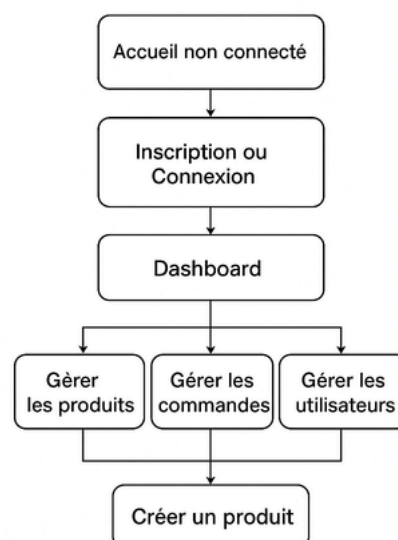
Arborescence

L'architecture Laravel + Inertia assure une séparation claire entre la logique serveur (Laravel) et la présentation (React), sans recourir à une API REST.

Parcours pour un User



Parcours pour un Admin

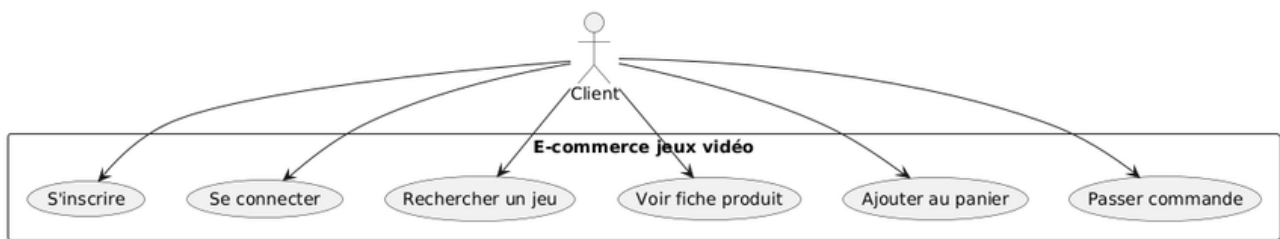


User Stories

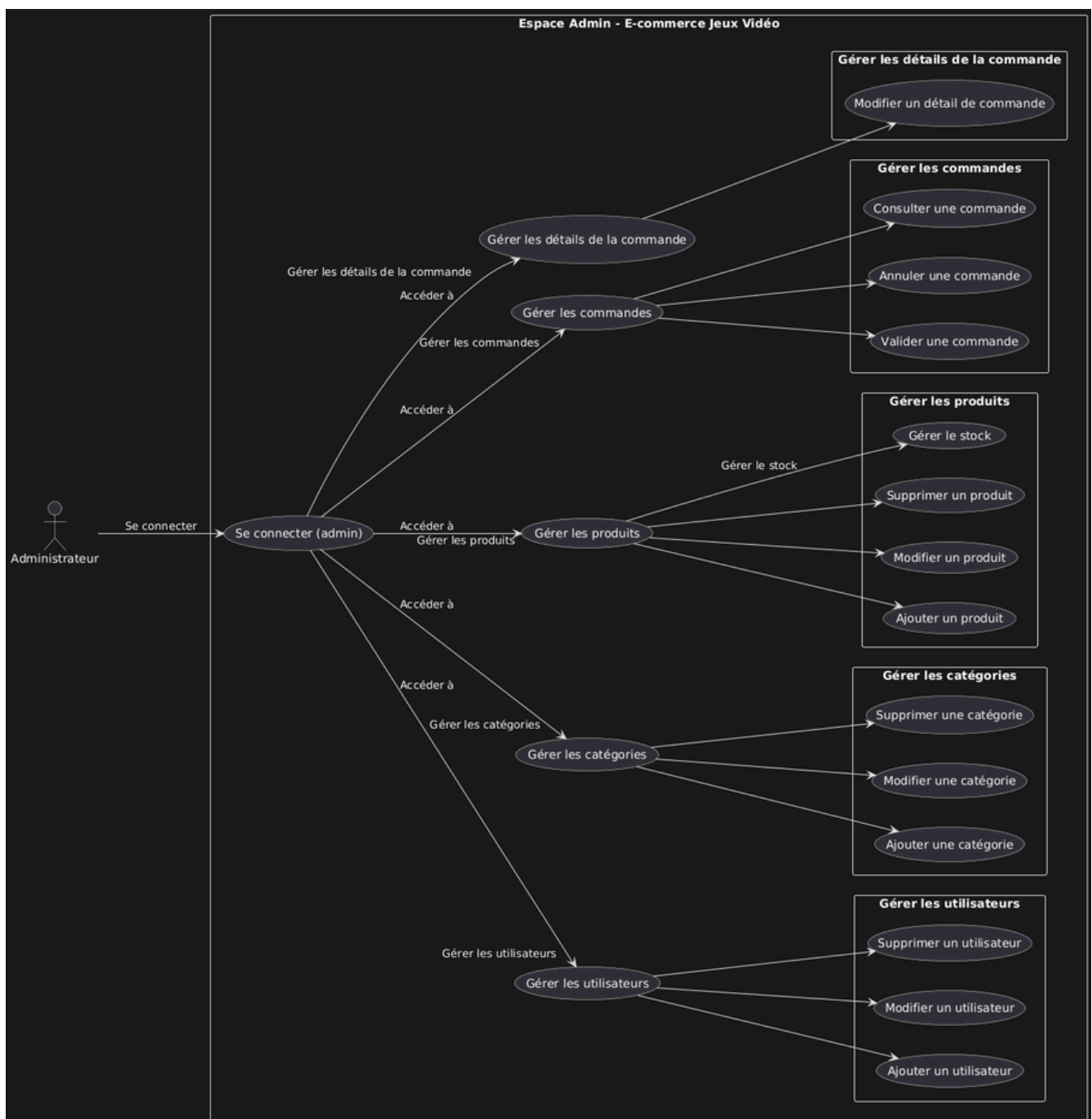
En tant que	Je souhaite	Afin de
Utilisateur	m'inscrire sur le site	pour accéder à mon compte.
Utilisateur	me connecter.	à mon espace personnel
Utilisateur	à mon espace personnel	modifier ou supprimer mon profil.
Utilisateur	consulter la liste des produits disponibles.	ajouter des produits à mon panier
Utilisateur	passer commande et payer mes achats.	passer commande et payer mes achats.
Administrateur	accéder à un espace d'administration sécurisé.	pour accéder à mon compte et au tableau bord
Administrateur	gérer la liste des utilisateurs .	(ajouter, modifier, supprimer)
Administrateur	gérer les produits	ajouter, modifier ou supprimer des produits.
Administrateur	gérer la liste des utilisateurs .	(ajouter, modifier, supprimer)

Les User Stories m'ont permis de définir clairement les besoins et les attentes des utilisateurs, tout en priorisant les fonctionnalités essentielles du projet Game Zone.

L'architecture client-serveur assure une séparation claire des responsabilités.



Présentation de l'architecture choisie pour l'admin

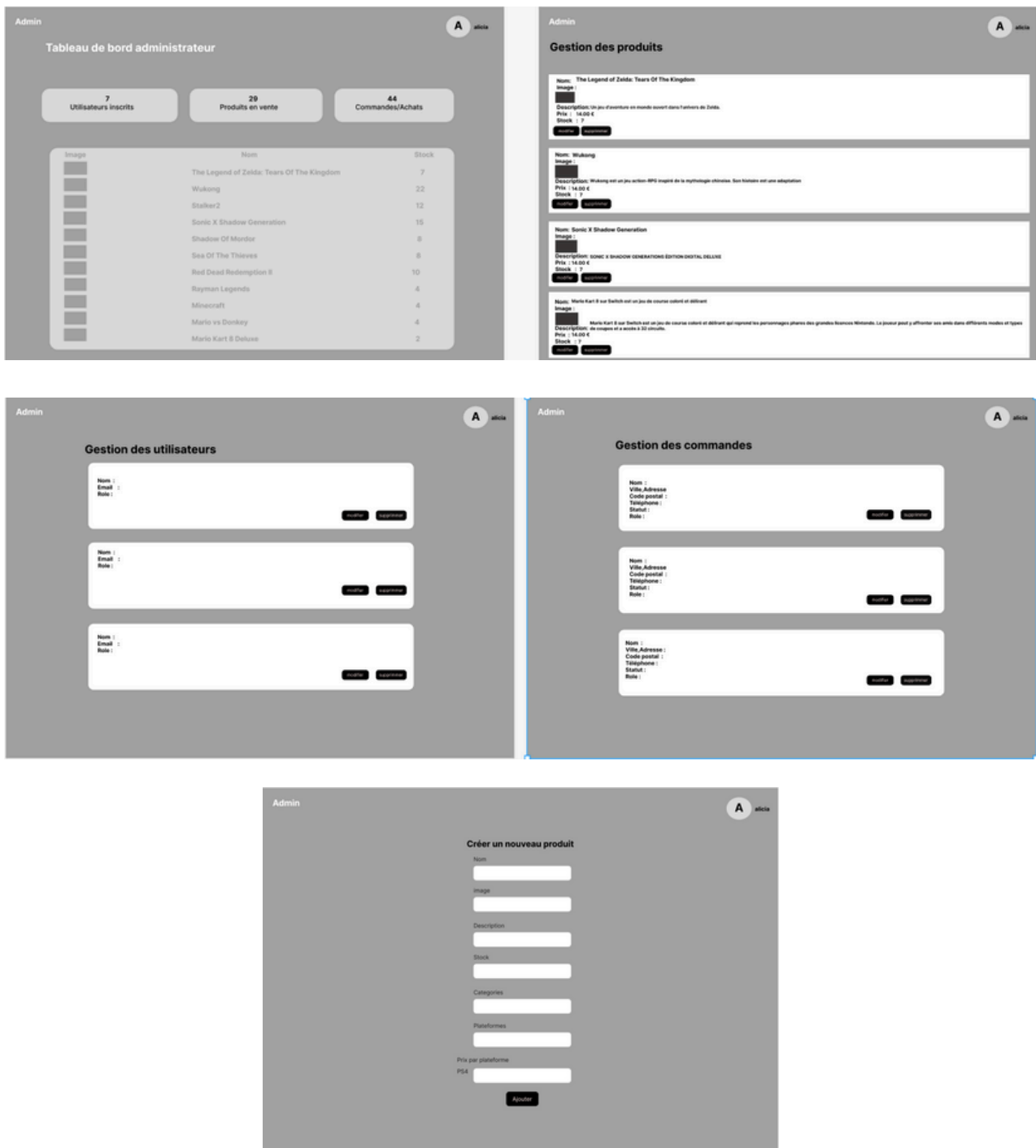


Wireframes côté Utilisateur

The wireframes represent the following user interface screens:

- Top Navigation:** A header bar with a circular profile icon, a search input field, and links for 'Connexion' and 'Inscription'.
- Créer un compte:** A registration form titled 'Créer un compte' with the subtitle 'Entrer vos informations'. It includes input fields for 'Nom', 'Adresse Mail', 'Mot De Passe', and 'Confirmer le mot de passe', followed by a 'S'inscrire' button.
- Connexion à votre compte:** A login form titled 'Connexion à votre compte' with the subtitle 'Entrez votre email et mot de passe pour vous connecter'. It features a 'Rôle' dropdown menu (set to 'Utilisateur'), input fields for 'Adresse Mail' and 'Mot De Passe', and a 'Se connecter' button.
- Product Listing:** A grid of eight product cards. Each card displays 'Nom', 'Prix', 'Categories', 'Plateformes disponibles', and a placeholder 'Image'.
- Product Detail:** A screen for a single product with a 'PSA' dropdown, 'Titre', 'Image du produit', 'Description', 'Prix' (set to '1'), and a 'Ajouter au Panier' button.
- Votre Panier:** A shopping cart interface with a table header: 'Image Produit', 'Nom', 'Quantité', 'Prix unitaire', and 'Prix total'. It includes a 'Continuer' button and a 'Retourner' link.
- Mes Commandes:** A list of orders, each showing 'Commande #47 - 2025-06-23T08:52:49.000000Z' and 'Total :'. The list is currently empty.
- Confirmation de votre commande:** A confirmation screen titled 'Confirmation de votre commande' with the subtitle 'Informations de livraison'. It contains input fields for 'Prénoms', 'Nom', 'Adresse', 'Ville', 'Code postal', 'Téléphone', and 'Numéro de carte (optionnel)', followed by a 'Confirmer et valider la commande' button. A 'Total : 40.00 €' is displayed in the top right.

Wireframes côté Admin

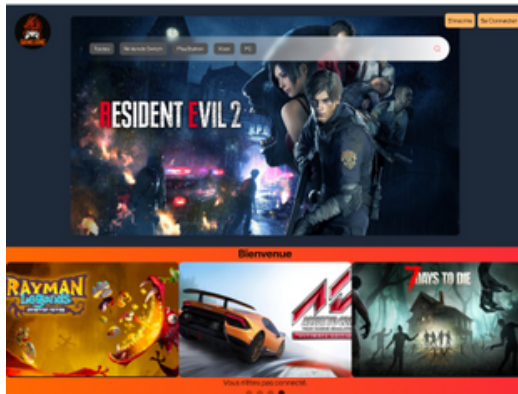


Pourquoi j'ai commencé par le wireframe

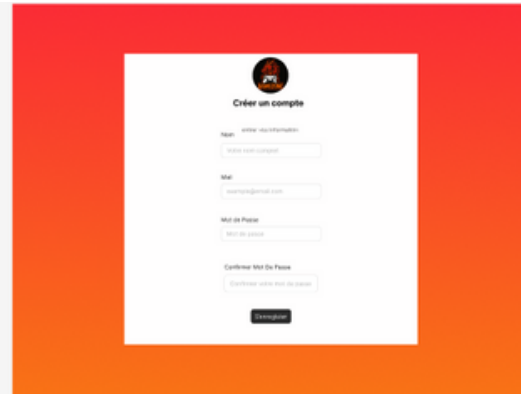
Le wireframe permet de poser rapidement la structure des pages (emplacement du menu, du contenu, des boutons...) sans se perdre dans les couleurs ou le design.

Maquette coté Utilisateur

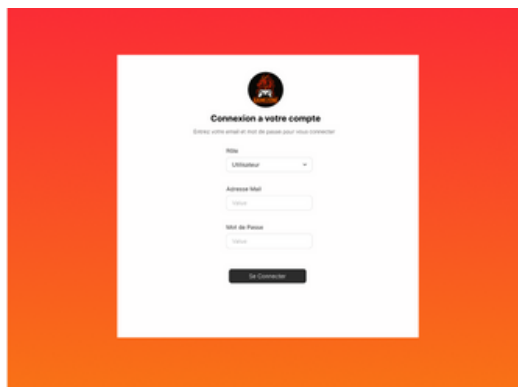
Accueil non Connecter



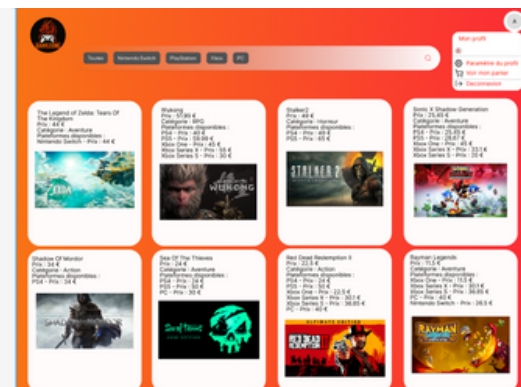
Inscription



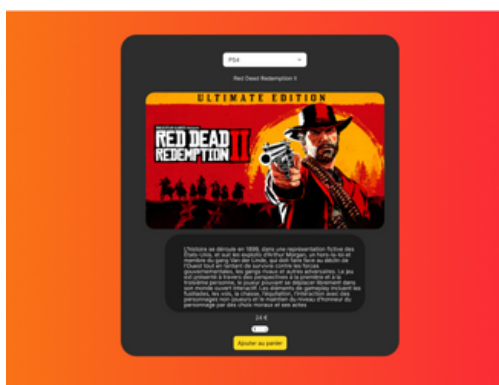
Connexion



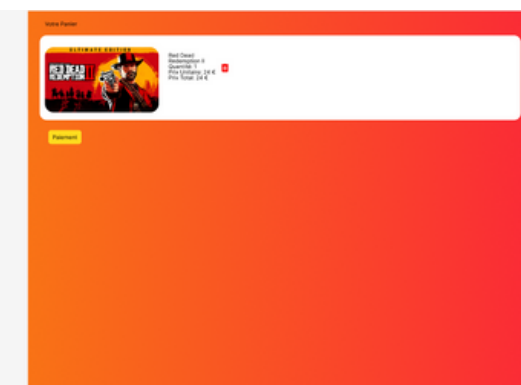
Accueil Connecter



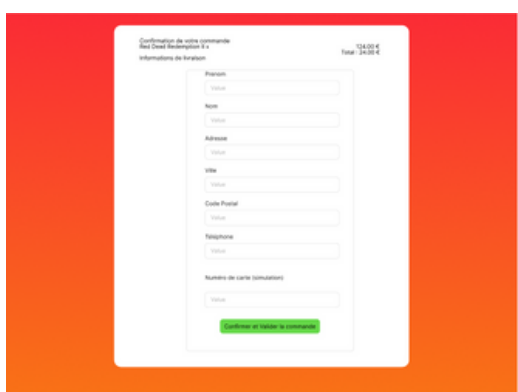
Détail Produit



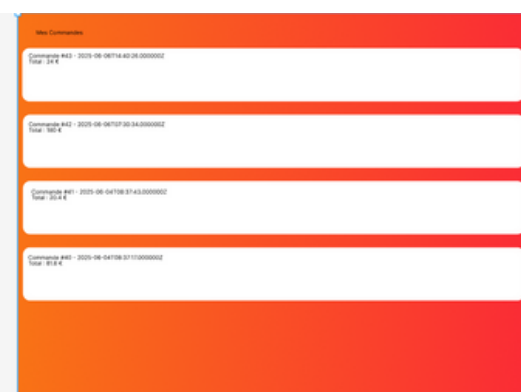
Panier



Paieiment

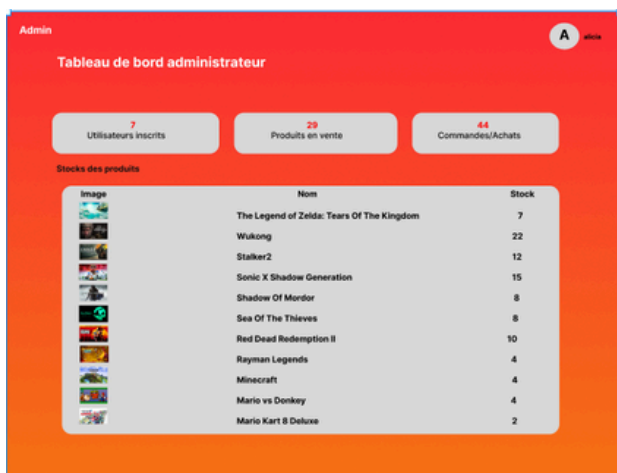


Mes commandes

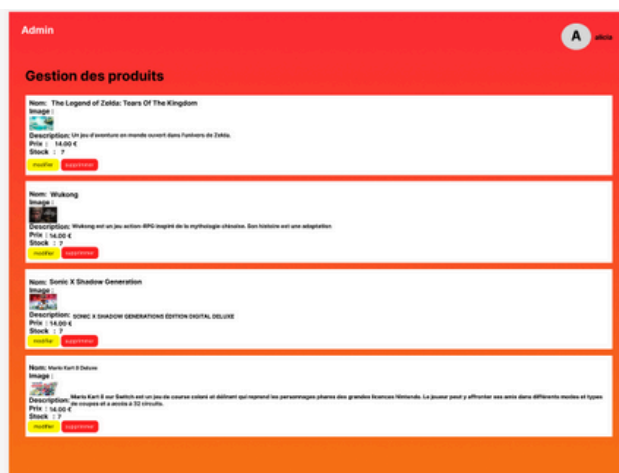


Maquette coté Admin

Dashboard



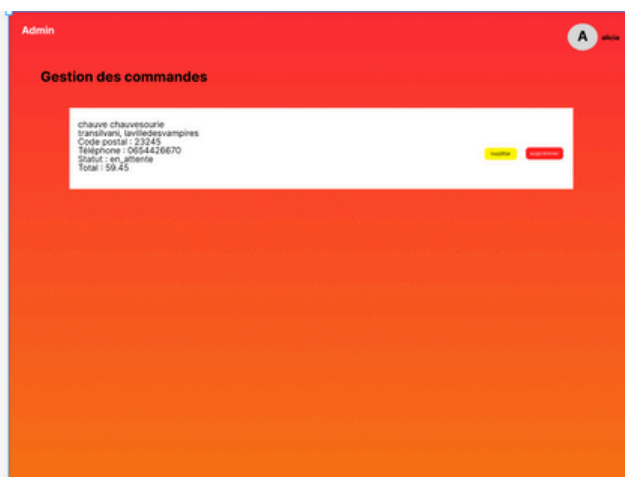
Gérer les produits



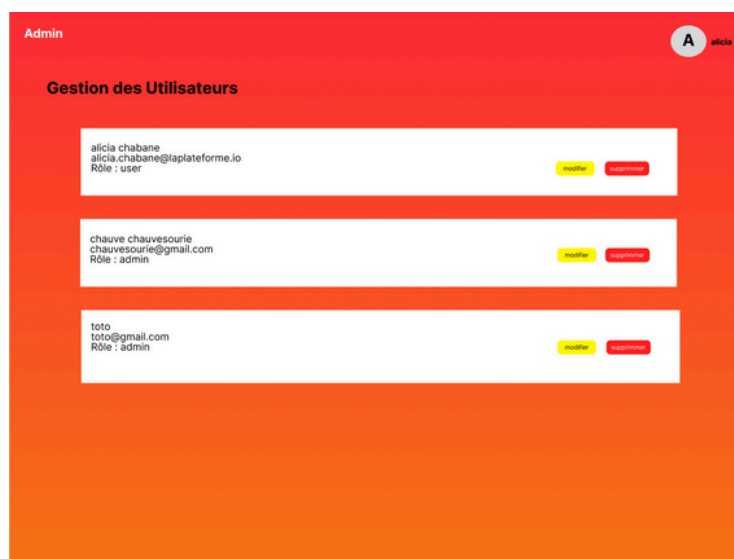
Créer un produit

A form for adding new products to the catalog. It includes input fields for Name, Image, Description, Stock, Categories, Platforms, Price per platform, and PEGI rating. An 'Ajouter' button is at the bottom.

Gérer les commandes



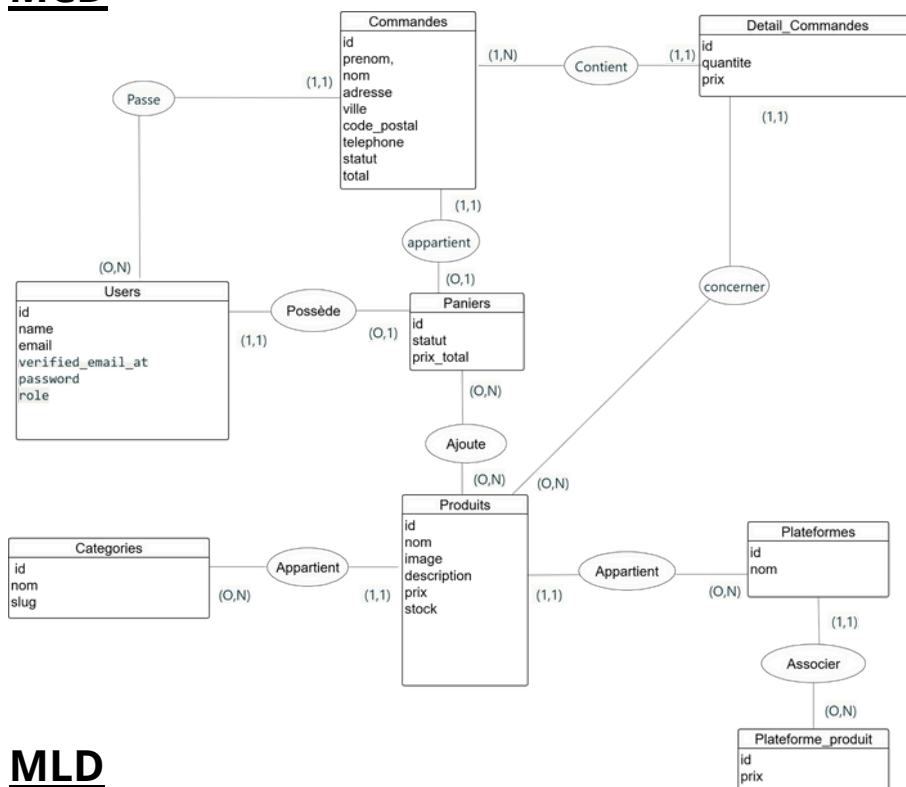
Gérer les utilisateurs



Je me suis inspirée du site Instant Gaming pour concevoir une maquette moderne et intuitive, reprenant les codes visuels du e-commerce dédié aux jeux vidéo.

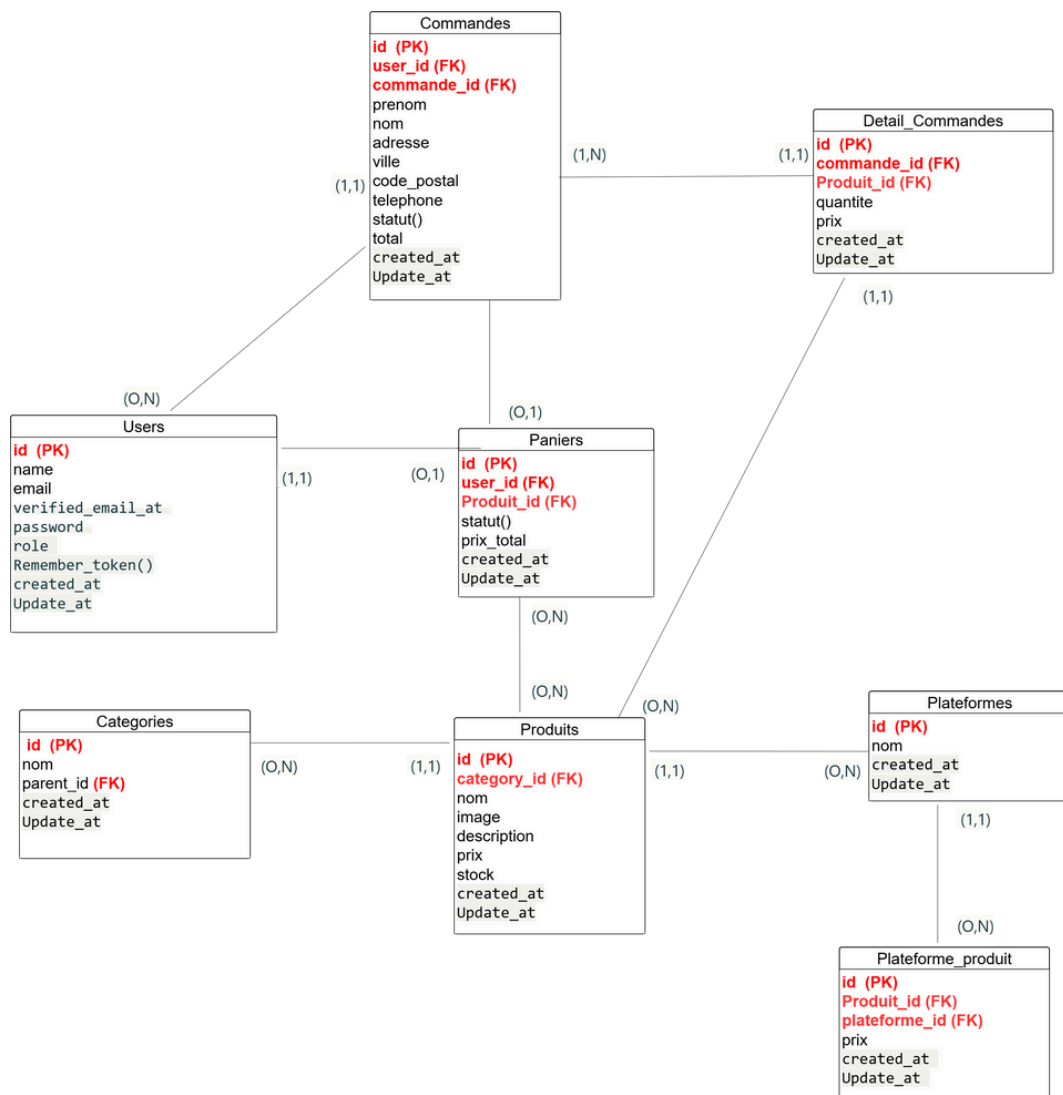
Merise

MCD

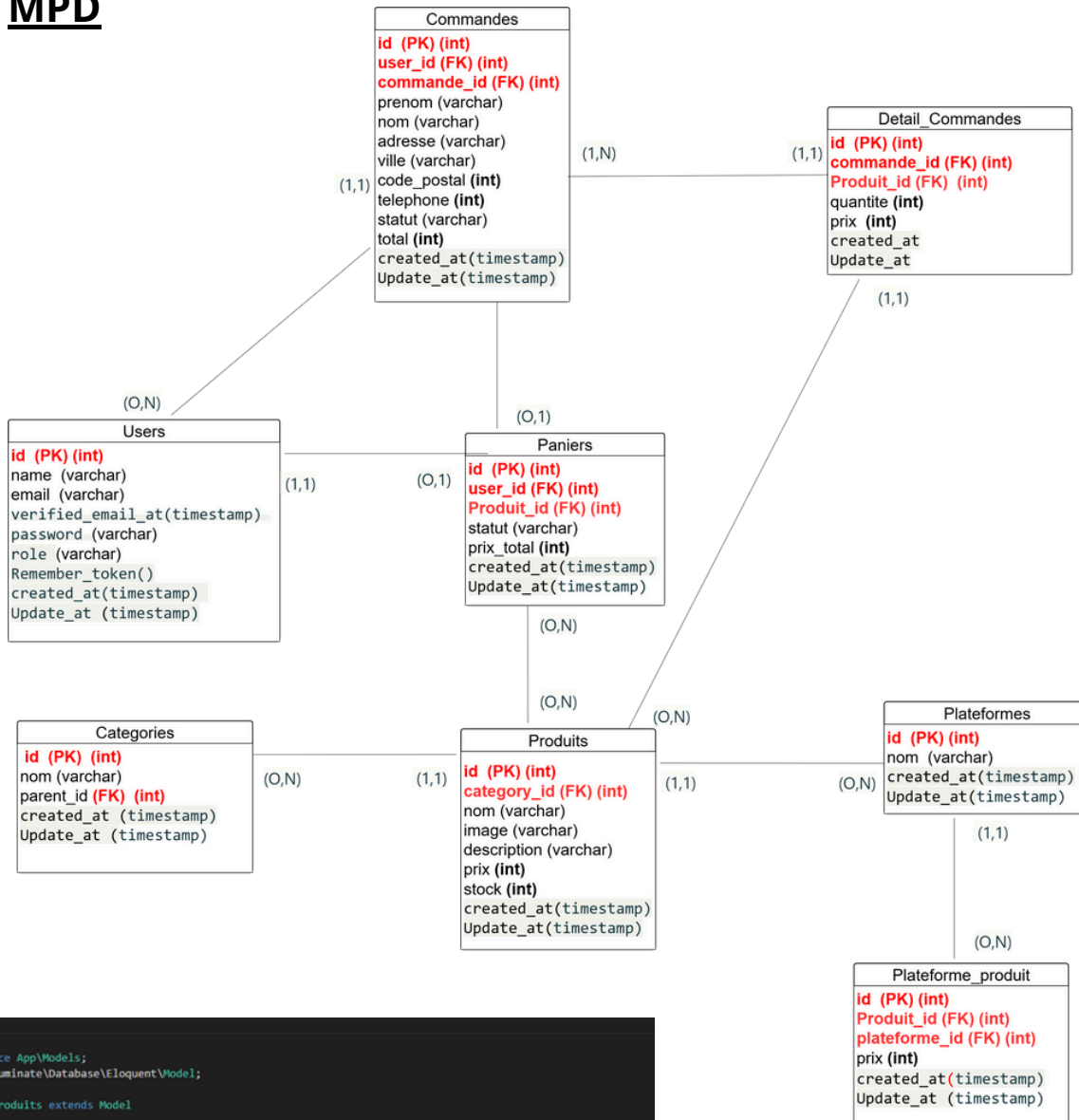


Pas de relation entre users et produit il s'exprime via la relation entre Utilisateur et Panier, puis entre Panier et Produit

MLD



MPD



```

<?php
namespace App\Models;
use Illuminate\Database\Eloquent\Model;

class Produits extends Model
{
    protected $table = 'produits';
    protected $fillable = ['category_id', 'nom', 'image', 'description', 'prix', 'stock', 'deleted_at'];
    public function category()
    {
        return $this->belongsTo(Categories::class, 'category_id');
    }

    public function plateformes()
    {
        return $this->belongsToMany(Plateforme::class, 'plateforme_produit', 'produit_id', 'plateforme_id')
            ->withPivot('prix')
            ->withTimestamps();
    }

    public function getPrixAttribute($value)
    {
        return $value;
    }

    public function scopeAvailable($query)
    {
        return $query->where('stock', '>', 0)->whereNull('deleted_at');
    }
}
    
```

```

1 <?php
2
3 use Illuminate\Database\Migrations\Migration;
4 use Illuminate\Database\Schema\Blueprint;
5 use Illuminate\Support\Facades\Schema;
6
7 class CreateProduitsTable extends Migration
8 {
9     public function up()
10     {
11         Schema::create('produits', function (Blueprint $table) {
12             $table->id();
13             $table->foreignId('category_id')->constrained()->onDelete('cascade');
14             $table->string('nom');
15             $table->string('image')->nullable();
16             $table->text('description')->nullable();
17             $table->decimal('prix', 10, 2);
18             $table->integer('stock')->default(0);
19             $table->timestamps();
20         });
21     }
22
23     public function down()
24     {
25         Schema::dropIfExists('produits');
26     }
27 }
    
```

Toutes les relations sont définies dans les migrations Laravel à l'aide de `foreignId()->constrained()` et `onDelete('cascade')`.

Le modèle Eloquent gère automatiquement les relations `belongsTo`, `hasMany` et `belongsToMany` entre les entités.

Réalisation en front

Formulaire d'inscription

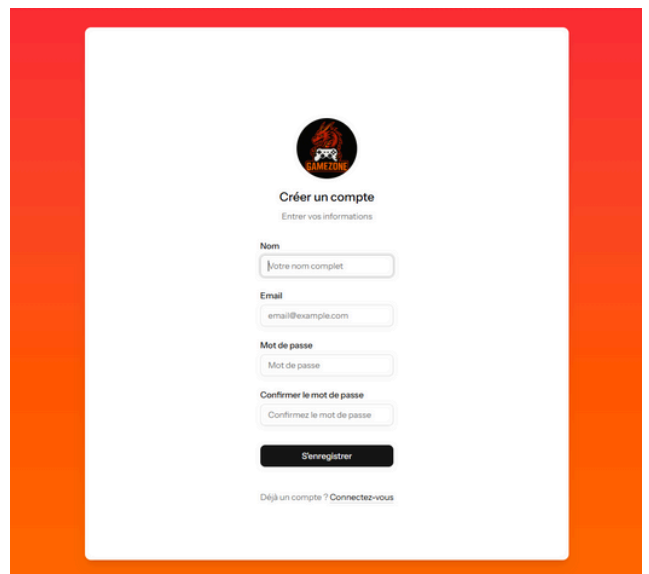
```
<AuthLayout title="Créer un compte" description="Entrer vos informations">
  <Head title="Register" />
  <form className="flex flex-col gap-6" onSubmit={submit}>
    <div className="grid gap-6">
      <div className="grid gap-2">
        <Label htmlFor="name">Nom</Label>
        <Input
          id="name"
          type="text"
          required
          autoFocus
          autoComplete="name"
          value={data.name}
          onChange={(e) => setData('name', e.target.value)}
          disabled={processing}
          placeholder="Votre nom complet"
        />
        <InputError message={errors.name} />
      </div>
      <div className="grid gap-2">
        <Label htmlFor="email">Email</Label>
        <Input
          id="email"
          type="email"
          required
          autoComplete="email"
          value={data.email}
          onChange={(e) => setData('email', e.target.value)}
          disabled={processing}
          placeholder="email@example.com"
        />
        <InputError message={errors.email} />
      </div>
```

```
      <div className="grid gap-2">
        <Label htmlFor="password">Mot de passe</Label>
        <Input
          id="password"
          type="password"
          required
          autoComplete="new-password"
          value={data.password}
          onChange={(e) => setData('password', e.target.value)}
          disabled={processing}
          placeholder="Mot de passe"
        />
        <InputError message={errors.password} />
      </div>
      <div className="grid gap-2">
        <Label htmlFor="password_confirmation">Confirmer le mot de passe</Label>
        <Input
          id="password_confirmation"
          type="password"
          required
          autoComplete="new-password"
          value={data.password_confirmation}
          onChange={(e) => setData('password_confirmation', e.target.value)}
          disabled={processing}
          placeholder="Confirmez le mot de passe"
        />
        <InputError message={errors.password_confirmation} />
      </div>
      <Button type="submit" className="mt-2 w-full" disabled={processing}>
        {processing && <LoaderCircle className="h-4 w-4 animate-spin mr-2" />}
        S'enregistrer
      </Button>
      <div className="text-muted-foreground text-center text-sm mt-4">
        Déjà un compte ?{' '}
        <TextLink href={route('login')}>
          Connectez-vous
        </TextLink>
      </div>
    </div>
  </form>
</AuthLayout>
}
```

ce composant React représente le formulaire d'inscription de l'application. Il utilise @inertiajs/react pour gérer les données du formulaire et l'envoi au serveur.

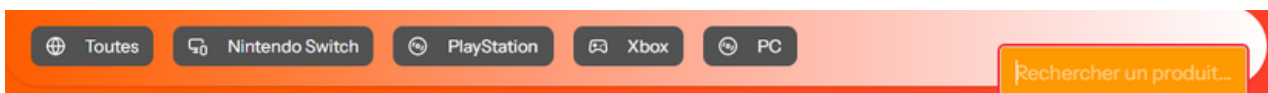
Les champs Nom, Email, Mot de passe et Confirmation du mot de passe sont obligatoires et gérés avec des messages d'erreur en cas de validation incorrecte.

Le bouton de soumission déclenche la méthode post(route('register')) qui envoie les données au backend. Pendant le traitement, le bouton est désactivé et affiche un loader. Enfin, un lien permet de rediriger vers la page de connexion si l'utilisateur possède déjà un compte.



Dans appLayout = NavMain

```
resources > js > layouts > app-layout.jsx > AppLayout
1 import { usePage } from '@inertiajs/react';
2 import { NavMain } from '@components/nav-main';
3 import { SidebarProvider } from '@components/ui/sidebar';
4
5 export default function AppLayout({ children }) {
6   const page = usePage();
7
8   const platformItems = [
9     { title: 'Toutes', icon: null },
10    { title: 'Nintendo Switch', icon: null },
11    { title: 'PlayStation', icon: null },
12    { title: 'Xbox', icon: null },
13    { title: 'PC', icon: null },
14  ];
15
16  const handlePlatformClick = (platform) => {
17
18  };
19
20  const handleSearch = (query) => {
21
22  };
23
24  return (
25    <SidebarProvider>
26    <div className="bg-gradient-to-r to-red-500 from-orange-500 w-full flex-col items-center justify-center relative overflow-x-hidden">
27
28      <header className="absolute top-10 left-0 w-full z-50">
29        <NavMain
30          items={platformItems}
31          onPlatformClick={handlePlatformClick}
32          onSearch={handleSearch}
33        />
34      </header>
35      <div /* Main pour le contenu */
36        <main className="">
37          {children}
38        </main>
39      </div>
40    </SidebarProvider>
41  );
42 }
```

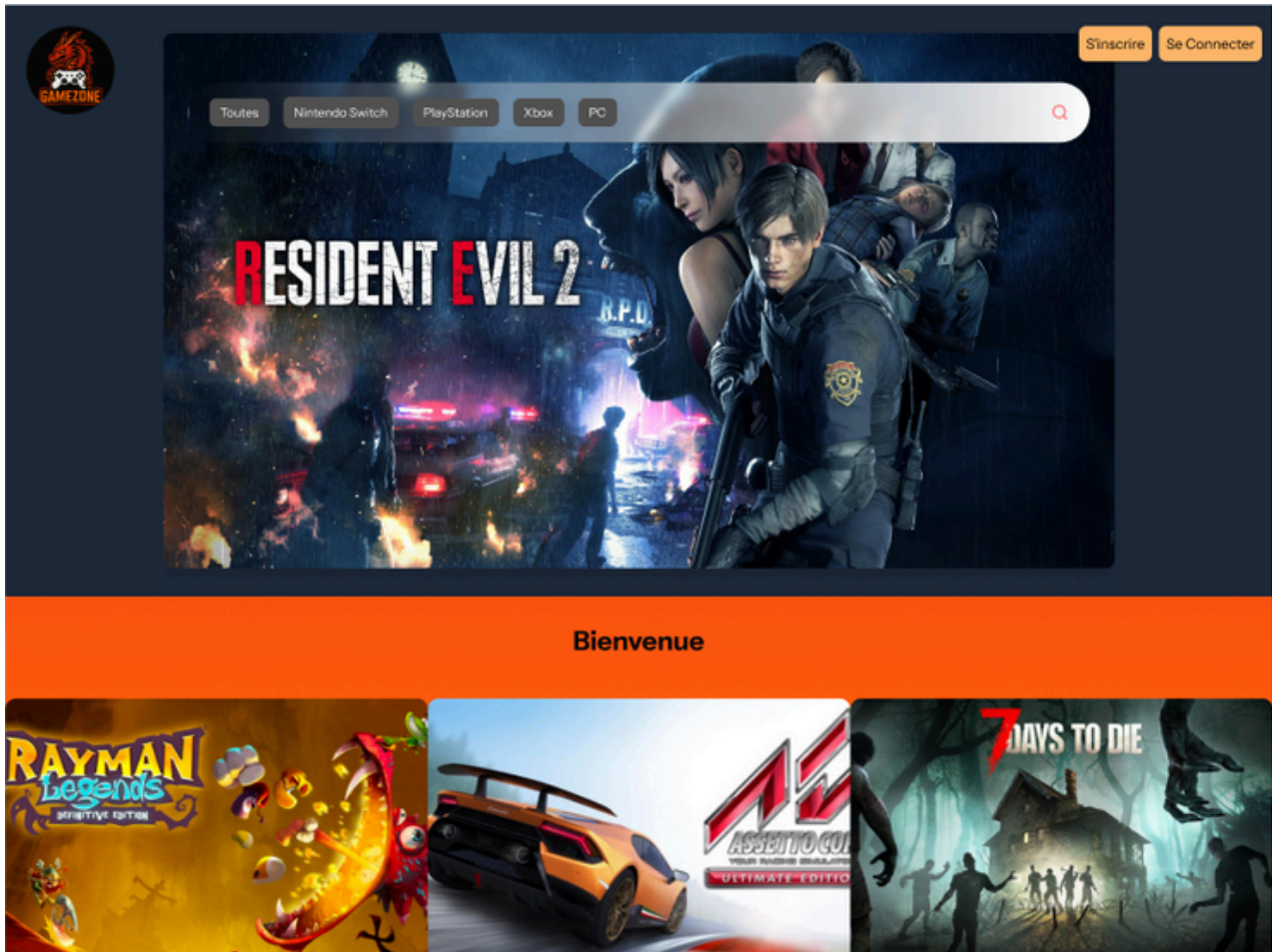


ce composant **AppLayout** :

- **Structure l'application globale** :
 - Fournit un **contexte Sidebar** via SidebarProvider.
 - Définit un **layout principal** avec un **header** et un **main** pour le contenu.
- **Intègre la barre de navigation principale (NavMain)** :
 - Passe une liste de plateformes (platformItems) comme éléments du menu.
 - Prépare des callbacks (handlePlatformClick, handleSearch) pour gérer les interactions.
- **Utilise usePage() (Inertia.js)** pour accéder aux données de la page en cours.
- **Applique un style visuel** avec Tailwind CSS (dégradé rouge/orange, position fixe du header, structure responsive).

Extraits de code UI statique (HTML/CSS)

```
<div className="w-full bg-gray-800 bg-cover flex justify-center items-start py-8">  
    
</div>
```



Cet extrait montre le code statique de ma page Accueil non connecter réalisée avec HTML et Tailwind CSS utilisée dans un composant React

Extraits de code UI dynamique (JS, React,)



Cet extrait de code illustre la gestion dynamique de l'interface utilisateur en React, permettant la mise à jour automatique des éléments affichés selon les actions de l'utilisateur, comme l'ajout d'un jeu au panier ou la connexion à un compte.

Adaptation responsive et accessibilité

j'utilise React et react-slick avec des paramètres "responsive"

```
npm install react-slick slick-carousel
```

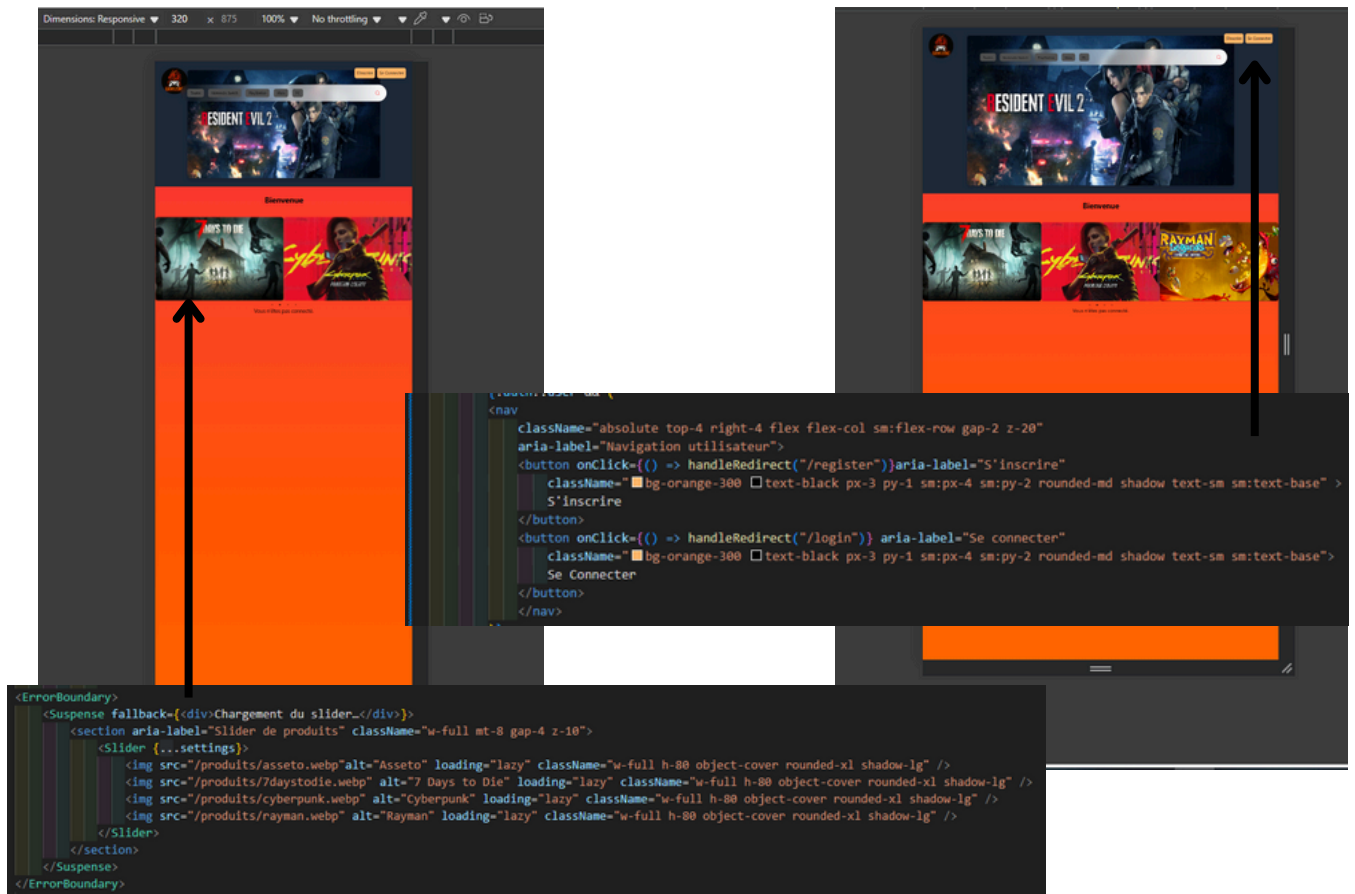
Import the required CSS files:

```
import "slick-carousel/slick/slick.css";  
import "slick-carousel/slick/slick-theme.css";
```

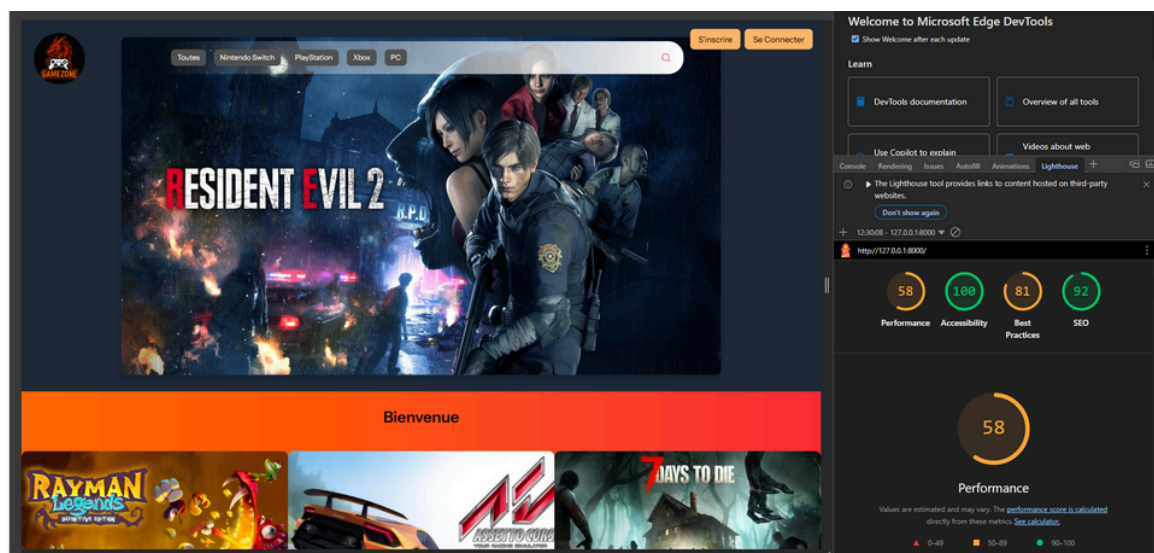
- avec Tailwind CSS

Mobile

Desktop



L'application est conçue pour être responsive grâce à Tailwind CSS et React, et respecte les bonnes pratiques d'accessibilité, notamment l'utilisation d'attributs alt pour les images, des contrastes adaptés et une navigation fluide sur tous les supports.



Réalisation migration

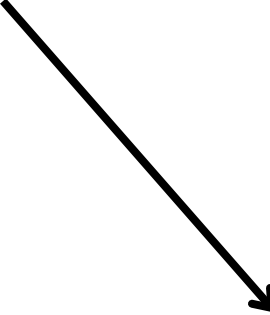
Table Produits

```
1  <?php
2
3  use Illuminate\Database\Migrations\Migration;
4  use Illuminate\Database\Schema\Blueprint;
5  use Illuminate\Support\Facades\Schema;
6
7  class CreateProduitsTable extends Migration
8  {
9      public function up()
10     {
11         Schema::create('produits', function (Blueprint $table) {
12             $table->id();
13             $table->foreignId('category_id')->constrained()->onDelete('cascade');
14             $table->string('nom');
15             $table->string('image')->nullable();
16             $table->text('description')->nullable();
17             $table->decimal('prix', 10, 2);
18             $table->integer('stock')->default(0);
19             $table->timestamps();
20         });
21     }
22
23     public function down()
24     {
25         Schema::dropIfExists('produits');
26     }
27 }
```

Réalisation models

Ce modèle représente la table **users** dans la base de données.

Il hérite de la classe `Authenticatable` de Laravel, ce qui lui permet de bénéficier de toutes les fonctionnalités liées à l'authentification (connexion, mot de passe sécurisé, gestion des sessions).



```
1  <?php
2
3  namespace App\Models;
4  use Illuminate\Database\Eloquent\Factories\HasFactory;
5  use Illuminate\Foundation\Auth\User as Authenticatable;
6  use Illuminate\Notifications\Notifiable;
7  use Illuminate\Support\Facades\Hash;
8
9  class User extends Authenticatable
10  {
11      use HasFactory, Notifiable;
12
13      protected $fillable = [
14          'name',
15          'email',
16          'password',
17          'role',
18          'created_at',
19          'updated_at',
20      ];
21
22      protected $hidden = [
23          'password',
24          'remember_token',
25      ];
26  }
```

Réalisation Controller

AuthController

```
class AuthController extends Controller
{
    public function register(Request $request)
    {
        $request->validate([
            'name' => 'required|string|max:255',
            'email' => 'required|email|unique:users',
            'password' => 'required|confirmed|min:6',
        ]);

        $user = User::create([
            'name' => $request->name,
            'email' => $request->email,
            'password' => Hash::make($request->password),
            'role' => 'user',
        ]);

        Auth::login($user);

        // Retour direct à l'accueil connecté utilisateur
        return redirect()->route('accueil.connecter');
    }
}
```

C'est un contrôleur Laravel (AuthController) qui gère l'inscription des utilisateurs.

Il utilise la méthode register pour valider les données reçues du formulaire d'inscription.

Il prend en charge deux rôles : user et admin.

Une fois l'inscription réussie, l'utilisateur est automatiquement connecté (Auth::login) et redirigé vers la page appropriée (dashboard pour admin, accueil.connecter pour utilisateur).

Accès aux données avec Eloquent ORM (sans API)

ProduitController

```
class ProduitsController extends Controller
{
    public function accueil()
    {
        $produits = Produits::with(['category', 'plateformes'])->get();
        return Inertia::render('AccueilConnecter', [
            'produits' => $produits,
            'auth' => ['user' => auth()->user()],
        ]);
    }
}
```

`$produits = Produits::with(['category', 'plateformes'])->get();` interroge la base de données pour récupérer tous les produits avec leurs relations "category" et "plateformes". Cette ligne interroge la base via Eloquent.

Ensuite, les données récupérées sont envoyées en réponse au frontend via `Inertia::render()`, ce qui rend la page avec ces données.

pas besoin de faire une requête avec SELECT*

Réalisation Seeders

CategoriesSeeders

```
database > seeders > CategoriesSeeder.php
1  <?php
2
3  namespace Database\Seeders;
4
5  use Illuminate\Database\Seeder;
6  use App\Models\Categories;
7
8  class CategoriesSeeder extends Seeder
9  {
10     public function run(): void
11     {
12
13         $categories = [
14             ['nom' => 'Aventure'],
15             ['nom' => 'RPG'],
16             ['nom' => 'Action'],
17             ['nom' => 'Plateforme'],
18             ['nom' => 'Horreur'],
19             ['nom' => 'Course'],
20         ];
21
22         foreach ($categories as $categorie) {
23             Categories::firstOrCreate($categorie);
24         }
25     }
26 }
27 }
```

Un seeder est une classe qui sert à insérer des données initiales ou de test dans la base de données automatiquement, ce qui facilite le développement et les tests de l'application. Ici, la classe CategoriesSeeder insère des catégories de jeux vidéo dans la table categories. La méthode run() définit un tableau avec des catégories (Aventure, RPG, Action, etc.) puis utilise la méthode firstOrCreate pour éviter les doublons et insérer uniquement si la catégorie n'existe pas déjà

Les seeders sont exécutés via la commande artisan dédiée php artisan db:seed,

Composants métier (logique serveur, POO)

AuthController : gestion de l'inscription, authentification, gestion des rôles

```
app > Http > Controllers > AuthController.php
1  <?php
2
3  namespace App\Http\Controllers;
4
5  use Inertia\Inertia;
6  use Illuminate\Http\Request;
7  use App\Models\User;
8  use Illuminate\Support\Facades\Auth;
9  use Illuminate\Support\Facades\Hash;
10
11 class AuthController extends Controller
12 {
13     public function register(Request $request)
14     {
15         $request->validate([
16             'name' => 'required|string|max:255',
17             'email' => 'required|email|unique:users',
18             'password' => 'required|confirmed|min:6',
19             'role' => 'nullable|in:user,admin',
20             'admin_code' => 'nullable|string|max:255|required_if:role,admin',
21         ]);
22
23         $role = 'user';
24         if ($request->role === 'admin' && $request->admin_code === env('ADMIN_REGISTRATION_CODE')) {
25             $role = 'admin';
26         }
27
28         $user = User::create([
29             'name' => $request->name,
30             'email' => $request->email,
31             'password' => ($request->password),
32             'role' => $role,
33         ]);
34
35         Auth::login($user);
36
37         if ($role === 'admin') {
38             return redirect()->route('dashboard');
39         } else {
40             return redirect()->route('accueil.connecter');
41         }
42     }
43 }
```

Modèle User : règles de validation, sécurité du mot de passe

```
27
28     protected function casts(): array
29     {
30         return [
31             'email_verified_at' => 'datetime',
32         ];
33     }
34
35     public function setPasswordAttribute($value)
36     {
37         if (!empty($value)) {
38             $this->attributes['password'] = Hash::make($value);
39         }
40     }
41
42     public function setEmailAttribute($value)
43     {
44         if (!empty($value)) {
45             $this->attributes['email'] = $value;
46         }
47     }
48 }
```


Sécurisation côté client

validation des formulaires,

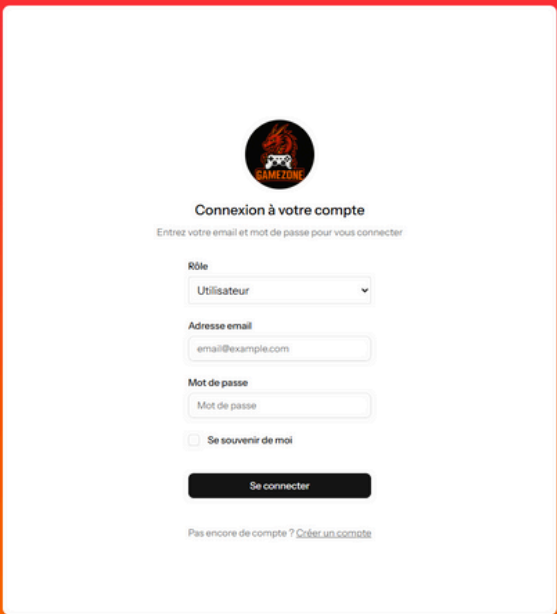
```
public function showLoginForm()  
{  
    return Inertia::render('auth/Login');  
}
```

resources/js/Pages/auth/Login.jsx

```
const submit = (e) => {  
    e.preventDefault();  
    post(route('login'), {  
        onFinish: () => reset('password'),  
    });  
};
```

```
public function loginSubmit(Request $request)  
{  
    $request->validate([  
        'email' => 'required|email',  
        'password' => 'required|min:6',  
    ]);  
  
    $credentials = $request->only('email', 'password');  
  
    if (Auth::attempt($credentials)) {  
        $request->session()->regenerate();  
  
        $user = Auth::user();  
  
        return $user->role === 'admin'  
            ? redirect()->route('dashboard')  
            : redirect()->route('accueil.connecter');  
    }  
  
    return Inertia::render('auth/Login', [  
        'error' => 'Email ou mot de passe incorrect.',  
    ]);  
}
```

affichage conditionnel,



The screenshot shows a login form for 'GAMIZONE'. At the top is a circular logo with a dragon and the text 'GAMIZONE'. Below the logo is the title 'Connexion à votre compte' and a subtitle 'Entrez votre email et mot de passe pour vous connecter'. The form contains a 'Rôle' dropdown menu with 'Utilisateur' selected, an 'Adresse email' input field with 'email@example.com', a 'Mot de passe' input field with 'Mot de passe', and a checkbox for 'Se souvenir de moi'. A 'Se connecter' button is at the bottom. A link 'Pas encore de compte ? Créer un compte' is at the very bottom.

confort utilisateur

Côté client, certaines vérifications et comportements visent avant tout à améliorer le confort utilisateur, sans remplacer les protections serveur.

Ces actions permettent de rendre l'application plus fluide, intuitive et réactive.

Sécurisation côté serveur (auth, validation,)

authentification

```
Route::get('/login', [AuthController::class, 'showLoginForm'])->name('login.form');
Route::get('/register', [AuthController::class, 'showRegisterForm'])->name('register.form');
Route::post('/login', [AuthController::class, 'loginSubmit'])->name('login');
Route::post('/register', [AuthController::class, 'register'])->name('register');
```

autorisation

```
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Support\Facades\Auth;

class AdminMiddleware
{
    public function handle($request, Closure $next)
    {
        if (!Auth::check() || Auth::user()->role !== 'admin') {
            abort(403, 'Accès refusé');
        }

        return $next($request);
    }
}
```

validation des données

```
public function store(Request $request)
{
    $validated = $request->validate([
        'nom' => 'required|string|max:255',
        'description' => 'required|string',
        'image' => 'required|image|mimes:jpg,jpeg,png|max:2048',
        'stock' => 'required|integer|min:0',
        'category_id' => 'required|exists:categories,id',
        'plateformes' => 'required|array',
        'plateformes.*' => 'exists:plateformes,id',
        'prix_plateformes' => 'required|array',
        'prix_plateformes.*' => 'required|numeric|min:0',
    ]);

    if ($request->hasFile('image')) {
        $imagePath = $request->file('image')->store('products', 'public');
        $validated['image'] = $imagePath;
    }

    $product = Products::create([
        'nom' => $validated['nom'],
        'description' => $validated['description'],
        'image' => $validated['image'],
        'prix' => 0,
        'stock' => $validated['stock'],
        'category_id' => $validated['category_id'],
    ]);

    $pivotData = [];
    foreach ($validated['plateformes'] as $plateformId) {
        $pivotData[$plateformId] = [
            'prix' => $validated['prix_plateformes'][$plateformId]
        ];
    }
    $product->plateformes()->sync($pivotData);

    return redirect()->route('products.index')->with('success', 'Produit créé avec succès !');
}
```

protection des routes. dans mon fichier web.php

```
Route::get('/dashboard', [AdminDashboardController::class, 'index'])
    ->middleware(['auth',])
    ->name('dashboard');

Route::middleware(['auth', AdminMiddleware::class])
    ->prefix('admin')
    ->name('admin.')
    ->group(function () {

        Route::get('/products', [ProductsController::class, 'index'])->name('products.index');
        Route::get('/products/creeur', [ProductsController::class, 'create'])->name('products.create');
        Route::post('/products', [ProductsController::class, 'store'])->name('products.store');
        Route::put('/products/{product}', [ProductsController::class, 'update'])->name('products.update');
        Route::delete('/products/{product}', [ProductsController::class, 'destroy'])->name('products.destroy');

        Route::get('/utilisateurs', [AdminDashboardController::class, 'users'])->name('utilisateurs.index');
        Route::put('/utilisateurs/{id}', [AdminDashboardController::class, 'update'])->name('utilisateurs.update');
        Route::delete('/utilisateurs/{id}', [AdminDashboardController::class, 'destroy'])->name('utilisateurs.destroy');

        Route::get('/commandes', [AdminDashboardController::class, 'commandes'])->name('commandes.index');
        Route::put('/commandes/{id}', [AdminDashboardController::class, 'update'])->name('commandes.update');
        Route::delete('/commandes/{id}', [AdminDashboardController::class, 'destroy'])->name('commandes.destroy');
    });
```

Pour sécuriser l'accès aux parties sensibles de mon application, notamment l'administration, j'ai utilisé le middleware auth de Laravel... les routes admin sont protégées par ce middleware

Test

```
1 </php>
2
3 namespace Tests\Feature\Auth;
4
5 use Illuminate\Foundation\Testing\RefreshDatabase;
6 use Tests\TestCase;
7 use App\Models\User;
8 use App\Providers\RouteServiceProvider;
9
10 class LoginTest extends TestCase
11 {
12     use RefreshDatabase;
13
14     public function test_login_screen_can_be_rendered()
15     {
16         $this->withoutExceptionHandling();
17         $response = $this->get('/login');
18         $response->assertStatus(200);
19     }
20
21     public function test_user_can_login_with_correct_credentials()
22     {
23         $this->withoutExceptionHandling();
24
25         $user = User::factory()->create([
26             'password' => bcrypt('password'),
27         ]);
28
29         $response = $this->post(route('login'), [
30             'email' => $user->email,
31             'password' => 'password',
32         ]);
33
34         dump($response->getContent());
35
36         $this->assertAuthenticatedAs($user);
37         $response->assertRedirect(RouteServiceProvider::HOME);
38     }
39 }
40
```

Test Login (fichier LoginTest.php)

```
PS C:\Users\Utilisateur\Desktop\E-commerceJeuVideo> php artisan test --filter=LoginTest

PASS Tests\Feature\Auth\LoginTest
✓ login screen can be rendered
✓ user can login with correct credentials

Tests: 2 passed (6 assertions)
Duration: 1.44s
```

. Test accès admin refusé (fichier AdminAccessTest.php)

```
8
9 class AdminAccessTest extends TestCase
10 {
11     use RefreshDatabase;
12
13     public function test_non_admin_cannot_access_admin_routes()
14     {
15         $user = User::factory()->create([
16             'role' => 'user',
17         ]);
18
19         $response = $this->actingAs($user)->get('/admin');
20
21         $response->assertStatus(403);
22     }
23 }
24
```

```
PS C:\Users\Utilisateur\Desktop\E-commerceJeuVideo> php artisan test

PASS Tests\Unit\ExampleTest
✓ that true is true

PASS Tests\Feature\AdminAccessTest
✓ non admin cannot access admin routes
✓ admin can access admin routes
```

Test parcours panier/commande (fichier CartAndOrderTest.php)

```
8 use App\Models\Product;
9
10 class CartAndOrderTest extends TestCase
11 {
12     use RefreshDatabase;
13
14     public function test_user_can_add_product_to_cart_and_place_order()
15     {
16         $user = User::factory()->create();
17         $product = Product::factory()->create();
18
19         $this->actingAs($user)
20             ->post('/cart/add', ['product_id' => $product->id])
21             ->assertStatus(200);
22
23         $response = $this->actingAs($user)
24             ->post('/order/checkout')
25             ->assertRedirect('/order/confirmation');
26
27         $this->assertDatabaseHas('orders', [
28             'user_id' => $user->id,
29         ]);
30     }
31 }
32
```

```
PASS Tests\Feature\CartAndOrderTest
✓ user can add produits to panier and place commandes

Tests: 1 passed (8 assertions)
Duration: 1.17s
```

```
PS C:\Users\Utilisateur\Desktop\E-commerceJeuVideo>
```

J'ai automatisé les tests principaux de mon application pour garantir la qualité et le bon fonctionnement.

J'ai vérifié que tout fonctionne avec la commande `php artisan test`

Présentation du jeu d'essai (fonctionnalité représentative)

Fonctionnalité testée : commande complète

Entrées : création de compte, ajout jeu au panier, passage commande

Données attendues : panier rempli, stock mis à jour, commande enregistrée, mail de confirmation envoyé

Données obtenues : tester dans la base, vérifier état base/commande/mail, captures d'écran explicatives.

Red Dead Redemption II

Prix : 22.5 €

Catégorie : Action

Plateformes disponibles :

PS4 - Prix : 24 €

PS5 - Prix : 50 €

Xbox One - Prix : 22.5 €

Xbox Series X - Prix : 30.1 €

Xbox Series S - Prix : 36.85 €

PC - Prix : 40 €



```
database > seeders > ProduitsSeeder.php
1 <?php
2
3 namespace Database\Seeders;
4
5 use Illuminate\Database\Seeder;
6 use App\Models\Produits;
7 use App\Models\Plateforme;
8 use App\Models\PlateformeProduit;
9
10 class ProduitsSeeder extends Seeder
11 {
12     public function run(): void
13     {
14         $produit = Produits::create([
15             'nom' => 'The Legend of Zelda: Tears Of The Kingdom',
16             'description' => 'Un jeu d\'aventure en monde ouvert dans l\'univers de Zelda.',
17             'image' => 'zelda.png',
18             'prix' => 34.00,
19             'stock' => 15,
20             'category_id' => 1,
21         ]);
22         $produit->plateformes()->attach([7]);
23
24         $produit = Produits::create([
25             'nom' => 'Mukong',
26             'description' => 'Mukong est un jeu action-RPG inspiré de la mythologie chinoise.
27             Son histoire est une adaptation des récits de La Pérégrination vers l\'Ouest,
28             l\'un des Quatre livres extraordinaires de la littérature chinoise.
29
30             Vous y incarnerez le Prédestiné et prendrez part à une épopée merveilleuse au cours
31             de laquelle vous devrez affronter maints périls pour découvrir la sombre vérité
32             d\'une légende glorieuse.
33
34             Explorez une terre regorgeant de merveilles.',
35             'image' => 'mukong.png',
36             'prix' => 59.99,
37             'stock' => 22,
38             'category_id' => 2,
39         ]);
40         $produit->plateformes()->attach([
41             1 => ['prix' => 40.00],
42             2 => ['prix' => 59.99],
43             3 => ['prix' => 45.00],
44             4 => ['prix' => 55.00],
45             5 => ['prix' => 30.00],
46         ]);
47     }
48 }
49
```



Rows: 7					
	id	nom	created_at	updated_at	
		Filter	Filter	Filter	Filter
1	1	PS4	NULL	NULL	
2	2	PS5	NULL	NULL	
3	3	Xbox One	NULL	NULL	
4	4	Xbox Series X	NULL	NULL	
5	5	Xbox Series S	NULL	NULL	
6	6	PC	NULL	NULL	
7	7	Nintendo Switch	NULL	NULL	
+	8				



Filter 1 rows... Upgrade to PRO									
id	user_id	produit_id	quantite	statut	prix	created_at	updated_at		
1	7	2	5	1	en_attente	34	2025-08-25	2025-08-25	

Cette capture d'écran illustre la table panier qui stocke les articles ajoutés par les utilisateurs. On y retrouve l'identifiant de l'utilisateur, l'identifiant du produit, la quantité, le prix et le statut de la commande (en_attente dans cet exemple). Cela prouve que la fonctionnalité d'ajout au panier et de suivi des commandes fonctionne correctement."

Filter 5 rows... Upgrade to PRO											
id	user_id	comm...	prenom	nom	adresse	ville	code_pos...	telephone			
1	1	2	NULL	chauve	chauvesourie	transilvani	lavilledesvampires	0008856	0654426670		
2	2	2	NULL	chauve	chauvesourie	transilvani	lavilledesvampires	0008856	0654426670		
3	3	2	NULL	chauve	chauvesourie	transilvani	lavilledesvampires	0008856	0654426670		
4	4	2	NULL	chocolat	pain	lesdessertsfameux	gateau	00234	0770554521		
5	5	2	NULL	chauve	chauvesourie	transilvani	lavilledesvampires	0008856	0654426670		

Confirmation de votre commande

Shadow Of Mordor x 1 34.00 €

Total : 34.00 €

Informations de livraison

Prenom
chauve

Nom
chauvesourie

Adresse
transilvani

Ville
lavilledesvampires

Code postal
0008856

Téléphone
0654426670

Numéro de carte (simulation)
1234 4564 3355 3421

Confirmer et valider la commande

```

PS C:\Users\Utilisateur\Desktop\le-commerce\leuvideo> php artisan migrate:fresh --seed
Dropping all tables ..... Is DONE
[INFO] Preparing database.
Creating migration table ..... 225.13ms DONE
[INFO] Running migrations.
0001_01_01_000000_create_users_table ..... 333.80ms DONE
0001_01_01_000001_create_cache_table ..... 13.05ms DONE
0001_01_01_000002_create_jobs_table ..... 133.46ms DONE
2025_07_05_071146_create_categories_table ..... 6.38ms DONE
2025_07_05_071244_create_products_table ..... 759.64ms DONE
2025_07_05_071315_create_commands_table ..... 10.74ms DONE
2025_07_05_071320_create_detail_commands_table ..... 9.25ms DONE
2025_07_05_071415_create_paniers_table ..... 20.73ms DONE
2025_07_05_071431_create_plateformes_table ..... 6.41ms DONE
2025_07_05_071454_create_plateforme_products_table ..... 17.40ms DONE
2025_09_02_082737_add_two_factor_columns_to_users_table ..... 44.81ms DONE
[INFO] Seeding database.
DatabaseSeeders\CategoriesSeeder ..... RUNNING
DatabaseSeeders\CategoriesSeeder ..... 377 ms DONE
DatabaseSeeders\PlateformeSeeder ..... RUNNING
DatabaseSeeders\PlateformeSeeder ..... 47 ms DONE
DatabaseSeeders\ProductsSeeder ..... RUNNING
DatabaseSeeders\ProductsSeeder ..... 628 ms DONE

```

Exécution de php artisan db:seed

Procédure d'installation et de configuration

J'ai commencé par installer les dépendances PHP nécessaires au projet à l'aide de Composer. Sur mon poste Windows, j'ai téléchargé et exécuté le fichier Composer-Setup.exe depuis le site officiel.

Avant cela, j'ai vérifié que PHP était bien installé sur ma machine, car Composer en dépend pour fonctionner correctement.

[Home](#) | [Getting Started](#) | [Download](#) | [Documentation](#) | [Browse Packages](#)

Download Composer Latest: v2.8.10

Windows Installer

The installer - which requires that you have PHP already installed - will download Composer for you and set up your PATH environment variable so you can simply call `composer` from any directory.

Download and run [Composer-Setup.exe](#) - it will install the latest composer version whenever it is executed.

installer Laravel : `composer create-project laravel/laravel nom-du-projet`

Installer React et les dépendances JS : `npm install react react-dom @inertiajs/react @vitejs/plugin-react`

installer Inertia.js côté Laravel : `composer require inertiajs/inertia-laravel`

configurer l'environnement .env

Créer le point d'entrée React : Dans `resources/js/app.jsx`

Adapter le layout Blade Dans `resources/views/app.blade.php`:

`@viteReactRefresh`: Hot reload
`@vite`: charge les fichiers JS/CSS
`@inertia`, `@inertiaHead`: points d'entrée Inertia

Lancer le projet

Backend Laravel : `php artisan serve`

Frontend React/Vite : `npm run dev`

Configurer Vite pour React : Dans `vite.config.js`

```
resources > views > app.blade.php
1 <!DOCTYPE html>
2 <html lang="{{ str_replace('_', '-', app()->getLocale()) }}">
3 <head>
4     <meta charset="utf-8">
5     <meta name="viewport" content="width=device-width, initial-scale=1">
6
7     <title inertia>{{ config('app.name', 'Laravel') }}</title>
8
9     @viteReactRefresh
10    @vite(['resources/css/app.css', 'resources/js/app.jsx'])
11    @inertiaHead
12 </head>
13 <body>
14     @inertia
15 </body>
16 </html>
```

```
vite.config.js
vite.config.js > ...
1 import tailwindcss from '@tailwindcss/vite';
2 import react from '@vitejs/plugin-react';
3 import laravel from 'laravel-vite-plugin';
4 import { resolve } from 'node:path';
5 import { defineConfig } from 'vite';
6 import path from 'path';
7 import { fileURLToPath } from 'url';
8
9 const __filename = fileURLToPath(import.meta.url);
10 const __dirname = path.dirname(__filename);
11
12 export default defineConfig({
13     plugins: [
14         laravel({
15             input: ['resources/css/app.css', 'resources/js/app.jsx'],
16             ssr: 'resources/js/ssr.jsx',
17             refresh: true,
18         }),
19         react(),
20         tailwindcss(),
21     ],
22 });
```

Routes

Route	Action	Controller @Méthode	Méthode http	Mildware	Rôle
/	Accueil non connecter		GET		Guest/User
/register	formulaire inscription	AuthController@ showRegisterForm	GET	guest	User
/register	inscription	AuthController @register	POST	guest	User
/login	formulaire connexion	AuthController @showLoginForm	GET	guest	Guest
/login	connexion	AuthController@ loginSubmit	POST	guest	Guest
/accueil- connecter	Accueil connecter		GET	auth, verified	User
/settings/ profile	voir son profil	ProfileController @edit	GET	auth	User
/recherche /produit	rechercher un produit	ProduitsController @search	GET		User/Guest
/produits	afficher les produits	ProduitsController @index	GET		User/Guest
/produits/{id}	Détail produit	ProduitsController @show	GET		User/Guest
/ajouter-au-panier	Ajouter au panier	PanierController@ ajouterAuPanier	POST	auth	User
/panier	afficher le panier	PanierController @afficher	GET	auth	User
/panier{id}	supprimer le panier	PanierController@ supprimer	DELETE	auth	User
/paiement	afficher page paiement	CommandeController @afficherPage Paiement	GET	auth	User
/paiement/valider	valider commande	CommandeController @validerCommande	POST	auth	User
/mes-commandes	afficher(liste) les commandes	CommandeController @mesCommandes	GET	auth	User
/logout	déconnexion	AuthController @logout	POST	auth	User

/dashboard	Dashboard	AdminDashboardController@index	GET	auth, Admin Middleware	Admin
/admin/products	Liste des produits	ProduitsController@index	GET	auth, Admin Middleware	Admin
/admin/store	Formulaire création produit	ProduitsController@create	GET	auth, Admin Middleware	Admin
/admin/products	Créer un produit	ProduitsController@store	POST	auth, Admin Middleware	Admin
/admin/products/{products}	Modifier un produit	ProduitsController@update	PUT	auth, Admin Middleware	Admin
/admin/products/{products}	Supprimer un produit	ProduitsController@destroy	DELETE	auth, Admin Middleware	Admin
/admin/user	Liste des utilisateurs	AdminDashboardController@users	GET	auth	Admin
/admin/user/{id}	Supprimer utilisateur	AdminDashboardController@destroy	DELETE	auth	Admin
/admin/user/{id}	Modifier utilisateur	AdminDashboardController@update	PUT	auth	Admin
/admin/commandes	Liste des commandes	AdminDashboardController@commandes	GET	auth	Admin
/admin/commandes/{id}	Mettre à jour commande	AdminDashboardController@update	PUT	auth	Admin
/admin/commandes/{commande}	Supprimer commande	AdminDashboardController@destroy	DELETE	auth	Admin

Ce tableau récapitule les routes, actions, contrôleurs et rôles nécessaires pour faciliter la gestion et la sécurité de l'application

Déploiement

Docker est utilisé pour simplifier le déploiement de l'application. Il permet de créer des conteneurs légers et portables contenant toutes les dépendances nécessaires, ce qui garantit que l'application fonctionne de la même manière sur toutes les machines (développement, test, production).

2. Structure du déploiement

```
FROM node:20
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Construction des images : docker build -t mon-app .

Lancement des conteneurs : docker-compose up -d

Description de la veille sur les vulnérabilités de sécurité

```
Route::middleware(['auth'])->prefix('admin')->group(function () {
    Route::get('/user', [AdminDashboardController::class, 'users'])->name('admin.user');
});
Route::delete('/admin/user/{id}', [AdminDashboardController::class, 'destroy'])->name('admin.user.destroy');
Route::put('/admin/user/{id}', [AdminDashboardController::class, 'update'])->name('admin.user.update');

Route::get('/admin/commandes', [AdminDashboardController::class, 'commandes'])->name('admin.commandes');

Route::put('/admin/commandes/{id}', [AdminDashboardController::class, 'update'])->name('admin.commandes.update');

Route::delete('/admin/commandes/{commande}', [AdminDashboardController::class, 'destroy'])->name('admin.commandes.destroy');
```

Pour sécuriser l'accès aux parties sensibles de mon application, notamment l'administration, j'ai utilisé le middleware auth de Laravel. Ce middleware est une protection standard qui s'assure qu'un utilisateur est authentifié avant d'accéder aux routes et contrôleurs protégés.

Dans le fichier de routes, j'ai regroupé les routes administratives sous un préfixe admin et appliqué le middleware auth à ce groupe :

Description d'une situation de recherche sur site anglophone

J'ai cherché sur des sites anglophones comme StackOverflow et la documentation officielle React comment gérer cette sécurité efficacement. J'ai découvert :

Que React échappe automatiquement les interpolations dans le JSX, empêchant la plupart des injections XSS.

Que pour insérer du HTML brut via la propriété `dangerouslySetInnerHTML`, il faut d'autant plus valider et nettoyer soigneusement les données

```
bootstrap > app.php
1  <?php
2
3  use App\Http\Middleware\HandleAppearance;
4  use App\Http\Middleware\HandleInertiaRequests;
5  use Illuminate\Foundation\Application;
6  use Illuminate\Foundation\Configuration\Middleware;
7  use Illuminate\Http\Middleware\AddLinkHeadersForPreloadedAssets;
8
9  return Application::configure(basePath: dirname(__DIR__))
10     ->withRouting(
11         web: __DIR__.'/../routes/web.php',
12         commands: __DIR__.'/../routes/console.php',
13         health: '/up',
14     )
15     ->withMiddleware(function (Middleware $middleware) {
16         $middleware->encryptCookies(except: ['appearance']);
17
18         // Activation de la protection CSRF automatique
19         $middleware->validateCsrfTokens();
20
21         $middleware->web(append: [
22             HandleAppearance::class,
23             HandleInertiaRequests::class,
24             AddLinkHeadersForPreloadedAssets::class,
25         ]);
26     })
27     ->withExceptions(function (Exceptions $exceptions) {
28         //
29     })->create();
30
```

Extrait du site anglophone + traduction Extrait anglais :

In Laravel 11, middleware configuration is typically done in the bootstrap/app.php file. You use the withMiddleware method to manage your application's middleware stack. This includes adding global middleware, middleware groups, and configuring CSRF protection. Unlike previous versions, the Kernel.php file is no longer used for middleware registration

Dans Laravel 11, la configuration des middlewares se fait généralement dans le fichier bootstrap/app.php. On utilise la méthode withMiddleware pour gérer la pile de middlewares de l'application. Cela inclut l'ajout de middlewares globaux, la configuration des groupes de middlewares et la protection CSRF. Contrairement aux versions précédentes, le fichier Kernel.php n'est plus utilisé pour l'enregistrement des middlewares.

Conclusion

Amélioration de l'interface utilisateur : Enrichissement du frontend avec de nouvelles fonctionnalités, animations et une meilleure expérience utilisateur.

Sécurité renforcée : Mise en place de mécanismes avancés (double authentification, audit des logs, suivi des vulnérabilités).